UNIVERSITÉ ALIOUNE DIOP DE BAMBEY

UFR Sciences & Technologies | Master 2 Big Data & Intelligence Artificielle

Année universitaire 2025-2026

PROJET TUTORÉ — CODES SOURCES COMPLETS

Optimisation Agricole Intelligente

Agri-Insight SN

Sécurité Alimentaire & Data-Driven Agriculture

Durée	4 semaines
Équipe	2 étudiants
Domaine	Big Data, Feature Engineering & Yield Prediction
Standard	M2 Approche Industrie / Sécurité Alimentaire
Enseignant	Mr Ahmed Ben Sidy Bouya SEYE — Senior Big Data & AI Engineer | Groupe Sonatel

Python/Faker • Pandera • NiFi • HDFS • Hive Bronze/Silver/Gold • PySpark MLlib • HBase • Airflow • MLflow

1. Contexte et Objectifs

L'agriculture sénégalaise contribue à 17% du PIB et emploie 60% de la population active. Ce projet vise à construire un pipeline industriel complet pour transformer des données agronomiques brutes en recommandations stratégiques pour la sécurité alimentaire — passant d'une agriculture de subsistance à une agriculture pilotée par la donnée (Data-Driven Agriculture).

> ℹ️ **Enjeu sécurité alimentaire**
> Dans un contexte de variabilité climatique croissante au Sahel, la capacité à prédire les rendements à partir des caractéristiques du sol et des données météorologiques est un levier stratégique. Agri-Insight SN fournit des recommandations concrètes : quelle culture, dans quelle région, pour quel type de sol, afin de maximiser la sécurité alimentaire.

1.1 Objectifs Techniques

#	Composant	Description
1	**Ingénierie Qualité**	Validation stricte des données de sol (pH, nutriments, pluviométrie) via Pandera — schéma avec contrainte agronomique Riz≥600mm/an
2	**Ingestion Orchestrée**	Flux NiFi GetFile → UpdateAttribute → PutHDFS vers couche Bronze
3	**Privacy by Design**	Anonymisation SHA-256 des parcelles et producteurs + drop des PII avant couche Silver
4	**Feature Engineering**	3 features dérivées : stress_hydrique, ph_optimal_score, indice_fertilite
5	**MLOps & Yield Prediction**	RandomForest MLlib — entraînement + inférence + monitoring drift Airflow + MLflow
6	**Mirroring & Analytics**	HBase (recommandations temps réel) + Hive Bronze/Silver/Gold (analyses historiques)

1.2 Architecture End-to-End

```
[ GÉNÉRATION ] ──(Pandera)──▶ [ NiFi ] ──▶ [ HDFS ] ──▶ [ SPARK ML ] ──▶ [ STORAGE ]
 Python/Faker     Validation    Orchest.     Bronze        Processing      HBase/HDFS
 Sols & Météo     Contrat       & Ingestion  Layer       Feature Eng.      Gold Layer
                                                        Yield Prediction       |
                                                              ▼              ▼
      [ MLOps LOOP ] ◀──────────── (Drift Detection) ◀──── [ ANALYTICS ]
       Airflow DAG                  MAE > 200 kg/ha          Apache Hive
       Réentraîn. hebdo.                                    Bronze / Silver / Gold
       MLflow Tracking                                       vue_recommandations
```

2. Prérequis & Installation

ℹ️ Configuration minimale

RAM : 16 Go. CPU : 4 cœurs. OS : Ubuntu 22.04 LTS. Docker Engine 24.x + Docker Compose 2.x. Java 11 (requis par Spark et HBase).

setup.sh

```
# Vérification environnement
java -version       # Attendu : openjdk 11.x
docker --version    # Docker 24.x minimum
free -h             # Minimum 14 Go disponibles

# Installation dépendances
python3.9 -m venv venv_agri && source venv_agri/bin/activate
pip install -r requirements.txt

# Démarrage infrastructure
mkdir -p data/bronze data/silver data/gold nifi_templates dags models
docker compose up -d zookeeper && sleep 10
docker compose up -d kafka nifi hbase hive-metastore namenode && sleep 30
docker compose up -d spark-master spark-worker airflow

# Initialiser HBase (une seule fois)
python hbase_setup.py

# Interfaces web :
#   NiFi      : http://localhost:8081
#   Spark     : http://localhost:8080
#   HBase     : http://localhost:16010
#   Airflow   : http://localhost:8082
#   HDFS      : http://localhost:9870
```

requirements.txt

```
# requirements.txt — Agri-Insight SN
# Testé : Python 3.9 / Java 11 / PySpark 3.3
pandas==2.2.3
numpy==2.1.2
faker==30.1.0
pandera==0.20.4
matplotlib==3.7.3
seaborn==0.12.2
scipy==1.11.4
happybase==1.2.0       # Client HBase Thrift
pyhive==0.7.0
thrift==0.16.0
apache-airflow==2.7.3
mlflow==2.8.1
```

3. Codes Sources Complets & Fonctionnels

3.1 Infrastructure Docker — docker-compose.yml

Infrastructure complète avec AGRI_SECRET_SALT injecté dans Spark, HDFS Namenode, et réseau dédié agri_net. Le Airflow volume monte les DAGs et les modèles.

docker-compose.yml

```yaml
version: '3.8'

services:
  zookeeper:
    image: bitnami/zookeeper:3.8
    container_name: zookeeper
    environment:
      ALLOW_ANONYMOUS_LOGIN: "yes"
    networks: [agri_net]

  kafka:
    image: bitnami/kafka:3.3
    container_name: kafka
    depends_on: [zookeeper]
    ports: ["9092:9092"]
    environment:
      KAFKA_CFG_ZOOKEEPER_CONNECT: zookeeper:2181
      KAFKA_CFG_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092
      ALLOW_ANONYMOUS_LOGIN: "yes"
      KAFKA_CFG_AUTO_CREATE_TOPICS_ENABLE: "true"
    networks: [agri_net]

  nifi:
    image: apache/nifi:1.23.2
    container_name: nifi
    ports: ["8081:8080"]
    volumes:
      - ./data/bronze:/opt/nifi/data/bronze
      - ./nifi_templates:/opt/nifi/nifi-current/conf/templates
    networks: [agri_net]

  spark-master:
    image: bitnami/spark:3.3
    container_name: spark-master
    environment:
      SPARK_MODE: master
      AGRI_SECRET_SALT: UADB_AGRI_2025
    ports: ["8080:8080", "7077:7077"]
    volumes:
      - ./models:/opt/models
    networks: [agri_net]

  spark-worker:
    image: bitnami/spark:3.3
    container_name: spark-worker
    depends_on: [spark-master]
    environment:
      SPARK_MODE: worker
      SPARK_MASTER_URL: spark://spark-master:7077
      AGRI_SECRET_SALT: UADB_AGRI_2025
    networks: [agri_net]

  hbase:
    image: harisekhon/hbase:2.1
    container_name: hbase
    ports: ["16010:16010", "9090:9090"]
    volumes:
      - hbase_data:/hbase-data
    networks: [agri_net]
```

```
  hive-metastore:
    image: apache/hive:3.1.3
    container_name: hive-metastore
    environment:
      SERVICE_NAME: metastore
    networks: [agri_net]

  airflow:
    image: apache/airflow:2.7.3
    container_name: airflow
    ports: ["8082:8080"]
    volumes:
      - ./dags:/opt/airflow/dags
      - ./models:/opt/airflow/models
    networks: [agri_net]

  namenode:
    image: bde2020/hadoop-namenode:2.0.0-hadoop3.2.1-java8
    container_name: namenode
    ports: ["9870:9870", "9000:9000"]
    environment:
      CLUSTER_NAME: agri_cluster
    networks: [agri_net]

volumes:
  hbase_data:
    driver: local

networks:
  agri_net:
    driver: bridge
```

3.2 Initialisation HBase — hbase_setup.py

> ⚠️ **Fichier obligatoire**
> Sans ce fichier, le DAG Airflow ne peut pas écrire les recommandations dans HBase. Crée les 3 tables : agri:recommandations, agri:predictions, agri:drift_monitor.

hbase_setup.py

```
# hbase_setup.py — Création tables HBase Agri-Insight SN
# Exécuter UNE SEULE FOIS après docker compose up

import happybase, logging
logger = logging.getLogger('HBaseSetup')

def create_agri_tables():
    conn = happybase.Connection('hbase', port=9090, timeout=10000)
    conn.open()
    tables = {
        # Recommandations temps réel par parcelle
        b'agri:recommandations': {
            b'meta':    {'max_versions': 1},   # region, culture, plot_id_secure
            b'predict': {'max_versions': 5},   # rendement_predit, score_confiance
            b'conseil': {'max_versions': 1},   # recommandation_texte, culture_optimale
        },
        # Historique prédictions (Gold Layer)
        b'agri:predictions': {
            b'input':   {'max_versions': 1},   # features utilisées
            b'output':  {'max_versions': 10},  # rendement_predit, modele_version
        },
        # Monitoring drift du modèle
        b'agri:drift_monitor': {
            b'stats':   {'max_versions': 30},  # 30 jours d'historique drift
        },
    }
    existantes = [t.decode() for t in conn.tables()]
    for nom_b, fam in tables.items():
        nom = nom_b.decode()
        if nom not in existantes:
            conn.create_table(nom_b, fam)
            logger.info(f'Table {nom} créée ✓')
        else:
            logger.info(f'Table {nom} déjà existante')
    conn.close()

if __name__ == '__main__':
    logging.basicConfig(level=logging.INFO)
    create_agri_tables()
```

3.3 Contrats Pandera — schema.py

> ✓ **Contrainte agronomique intégrée**
> Le schéma inclut une validation métier : le Riz exige au moins 600mm/an de pluviométrie. Les parcelles Riz avec pluvio < 600mm sont rejetées avec log d'avertissement, reflétant les contraintes réelles de l'agriculture sénégalaise.

schema.py

```
# schema.py — Contrats Pandera Agri-Insight SN
# Validation des contraintes agronomiques sénégalaises

import pandera as pa
from pandera.typing import Series
```

```
import pandas as pd, logging

logger = logging.getLogger('AgriSchema')

# Régions couvertes (7 régions agricoles clés du Sénégal)
REGIONS = ['Dakar','Thies','Kaolack','Ziguinchor',
           'Saint-Louis','Louga','Tambacounda']
CULTURES = ['Mil','Riz','Arachide','Manioc','Niebe']
TYPES_SOL = ['Sablonneux','Argileux','Limoneux','Ferralitique','Dior']

class AgriDataSchema(pa.SchemaModel):
    """Contrat de données : parcelles agricoles sénégalaises."""
    parcel_id:                   Series[str]   = pa.Field(unique=True)
    producer_name:               Series[str]   = pa.Field()  # Sera anonymisé
    region:                      Series[str]   = pa.Field(isin=REGIONS)
    culture:                     Series[str]   = pa.Field(isin=CULTURES)
    type_sol:                    Series[str]   = pa.Field(isin=TYPES_SOL)
    ph_sol:                      Series[float] = pa.Field(ge=4.5, le=8.5)
    pluviometrie_annuelle:       Series[float] = pa.Field(ge=200, le=1500)
    temperature_moy_celsius:     Series[float] = pa.Field(ge=15.0, le=45.0)
    humidite_relative_pct:       Series[float] = pa.Field(ge=10.0, le=100.0)
    teneur_matiere_organique_pct:Series[float] = pa.Field(ge=0.1, le=5.0)
    surface_hectares:            Series[float] = pa.Field(gt=0.0, le=5000.0)
    annee:                       Series[int]   = pa.Field(ge=2018, le=2030)
    rendement_kg_ha:             Series[float] = pa.Field(ge=0.0)

    # Cohérence agronomique : Riz exige au moins 600mm/an
    @pa.check('pluviometrie_annuelle', name='riz_needs_water')
    def check_riz_pluvio(cls, series, culture):
        riz_mask = (culture == 'Riz')
        return ~riz_mask | (series >= 600)

    class Config:
        coerce = True
        strict = True

def validate_and_filter(df: pd.DataFrame) -> pd.DataFrame:
    """Valide et filtre les lignes invalides en loguant les erreurs."""
    try:
        return AgriDataSchema.validate(df, lazy=True)
    except pa.errors.SchemaErrors as exc:
        err = exc.failure_cases
        logger.warning(f'[AgriDataSchema] {len(err)} erreur(s) rejetées')
        valid_idx = df.index.difference(err['index'].dropna().astype(int))
        return df.loc[valid_idx]
```

3.4 Simulateur de Données — generate_agri_data.py

Génère des parcelles agricoles réalistes avec des paramètres calés sur chaque région (Louga : sol sec/peu de pluie → Mil/Arachide, Ziguinchor : sol humide → Riz/Manioc). La fonction compute_rendement() utilise les mêmes formules que le LAB01.

generate_agri_data.py

```
# generate_agri_data.py — Simulateur données parcelles agricoles
# Génère des données réalistes avec saisonnalité sénégalaise
import pandas as pd, numpy as np, random, uuid
from faker import Faker
from schema import validate_and_filter

fake = Faker('fr_FR')
random.seed(42)
np.random.seed(42)

REGIONS_PARAMS = {
    'Louga':       {'pluvio': (350, 80),  'temp': (28, 2), 'cultures':
['Mil','Arachide']},
    'Saint-Louis': {'pluvio': (300, 60),  'temp': (27, 2), 'cultures': ['Riz','Mil']},
    'Kaolack':     {'pluvio': (700, 100), 'temp': (29, 2), 'cultures':
['Arachide','Mil','Niebe']},
    'Ziguinchor':  {'pluvio': (1200,150), 'temp': (27, 1), 'cultures':
['Riz','Manioc','Arachide']},
    'Thies':       {'pluvio': (500, 80),  'temp': (26, 2), 'cultures':
['Arachide','Mil']},
    'Dakar':       {'pluvio': (430, 70),  'temp': (25, 2), 'cultures':
['Arachide','Mil']},
    'Tambacounda': {'pluvio': (900, 120), 'temp': (30, 2), 'cultures':
['Mil','Manioc','Niebe']},
}

RENDEMENT_BASE = {
    'Mil':800, 'Riz':2500, 'Arachide':1000, 'Manioc':8000, 'Niebe':700
}

def compute_rendement(culture, pluvio, temp, ph, mo) -> float:
    """Rendement simulé selon les facteurs agronomiques."""
    base = RENDEMENT_BASE[culture]
    # Sensibilité pluviométrie
    k_pluvio = {'Mil':0.7,'Riz':1.0,'Arachide':0.6,'Manioc':0.4,'Niebe':0.5}[culture]
    # Sensibilité température (optimum ~27°C)
    k_temp   = {'Mil':-0.1,'Riz':-0.15,'Arachide':-0.05,'Manioc':-0.05,'Niebe':-
0.08}[culture]
    r = base
    r *= (1 + k_pluvio * (pluvio - 800) / 800)
    r *= (1 + k_temp   * (temp   - 27)  / 5)
    r *= (1 + 0.2      * (mo     - 1.5) / 1.5)
    r *= random.uniform(0.92, 1.08)  # Variabilité naturelle
    return max(50.0, round(r, 1))

def generate_parcelles(n_par_region: int = 100) -> pd.DataFrame:
    """Génère les données de parcelles pour toutes les régions."""
    rows = []
    for region, params in REGIONS_PARAMS.items():
        for year in range(2018, 2024):
            for _ in range(n_par_region):
                culture = random.choice(params['cultures'])
                pluvio  = max(200, round(np.random.normal(*params['pluvio']), 1))
                temp    = round(np.random.normal(*params['temp']), 1)
                ph      = round(random.uniform(5.0, 8.0), 1)
                mo      = round(random.uniform(0.5, 3.0), 2)
                rows.append({
                    'parcel_id':                    str(uuid.uuid4()),
                    'producer_name':                fake.name(),  # Sera anonymisé
                    'region':                       region,
                    'culture':                      culture,
                    'type_sol':                     random.choice(
```

```
['Sablonneux','Argileux','Limoneux',
                                                        'Ferralitique','Dior']),
                    'ph_sol':                       ph,
                    'pluviometrie_annuelle':        pluvio,
                    'temperature_moy_celsius':      temp,
                    'humidite_relative_pct':        round(random.uniform(30, 90), 1),
                    'teneur_matiere_organique_pct': mo,
                    'surface_hectares':             round(random.uniform(0.5, 50.0), 2),
                    'annee':                        year,
                    'rendement_kg_ha':
compute_rendement(culture,pluvio,temp,ph,mo),
                })
    df = pd.DataFrame(rows)
    df = validate_and_filter(df)  # Validation Pandera
    df.to_csv('data/bronze/agri_parcelles_brut.csv', index=False)
    print(f'Généré : {len(df)} parcelles ✓')
    return df

if __name__ == '__main__':
    import os; os.makedirs('data/bronze', exist_ok=True)
    generate_parcelles(n_par_region=100)
    # Attendu : ~4 200 lignes (7 régions × 6 ans × 100 parcelles)
```

3.5 Pipeline Spark — yield_prediction.py

⚠️ **Points clés du pipeline**

Privacy : drop('parcel_id', 'producer_name') après SHA-256 — PII jamais en Silver/Gold.
3 features dérivées : stress_hydrique (ratio pluvio/besoin), ph_optimal_score, indice_fertilite.
Mode --mode train : entraîne et sauvegarde le modèle dans HDFS.
Mode --mode infer : charge le modèle et écrit les prédictions dans predictions_gold.
COALESCE sur pluviometrie_annuelle et rendement_kg_ha protège contre les NULL résiduels.

yield_prediction.py

```
# yield_prediction.py — Privacy Layer + Feature Engineering + MLlib
# Pipeline Bronze → Silver → Gold + Prédiction rendement

import os
from pyspark.sql import SparkSession
from pyspark.sql.functions import (
    col, sha2, concat, lit, coalesce, when
)
from pyspark.sql.types import DoubleType
from pyspark.ml.feature import (
    StringIndexer, OneHotEncoder, VectorAssembler,
    StandardScaler, Imputer
)
from pyspark.ml.regression  import RandomForestRegressor
from pyspark.ml.evaluation  import RegressionEvaluator
from pyspark.ml             import Pipeline, PipelineModel
import argparse

SALT      = os.environ.get('AGRI_SECRET_SALT', 'default_salt')
MODEL_PATH= 'hdfs:///agri_insight/models/yield_rf_latest'

parser = argparse.ArgumentParser()
parser.add_argument('--mode', choices=['train','infer'], default='train')
parser.add_argument('--input-table', default='agri_insight.parcelles_silver')
args = parser.parse_args()

spark = SparkSession.builder \
    .appName('AgriInsight_YieldPrediction') \
    .config('spark.sql.shuffle.partitions', '8') \
    .enableHiveSupport() \
    .getOrCreate()
spark.sparkContext.setLogLevel('WARN')

# ── Bronze → Silver : Privacy Layer ──────────────────────────────────
raw_df = spark.table(args.input_table)

# CORRECTION : drop('parcel_id', 'producer_name') après anonymisation
silver_df = (
    raw_df
    .withColumn('plot_id_secure',
                sha2(concat(col('parcel_id'), lit(SALT)), 256))
    .drop('parcel_id', 'producer_name')  # Suppression OBLIGATOIRE des PII
)

# ── Silver → Gold : Feature Engineering ──────────────────────────────
# Feature 1 : Indice de stress hydrique (pluvio vs besoin culture)
BESOIN_PLUVIO = {'Mil':600.0,'Riz':1000.0,'Arachide':700.0,
                 'Manioc':800.0,'Niebe':500.0}
bc_besoin = spark.sparkContext.broadcast(BESOIN_PLUVIO)

from pyspark.sql.functions import udf
from pyspark.sql.types import FloatType

@udf(returnType=FloatType())
def stress_hydrique(culture, pluvio):
    """Ratio pluviométrie réelle / besoin théorique de la culture."""
    besoin = bc_besoin.value.get(culture, 700.0)
    return float(min(2.0, max(0.0, pluvio / besoin)))
```

```
gold_df = (
    silver_df
    # Feature dérivée 1 : stress hydrique
    .withColumn('stress_hydrique', stress_hydrique(col('culture'),
col('pluviometrie_annuelle')))
    # Feature dérivée 2 : pH optimal (1 = optimal, 0 = dégradé)
    .withColumn('ph_optimal_score',
        when((col('ph_sol') >= 6.0) & (col('ph_sol') <= 7.5), lit(1.0))
        .when((col('ph_sol') >= 5.5) & (col('ph_sol') <= 8.0), lit(0.7))
        .otherwise(lit(0.3)))
    # Feature dérivée 3 : indice de fertilité composite
    .withColumn('indice_fertilite',
        col('teneur_matiere_organique_pct') * col('ph_optimal_score'))
    # Protection contre les NULL résiduels
    .withColumn('pluviometrie_annuelle',
                coalesce(col('pluviometrie_annuelle'), lit(600.0)))
    .withColumn('rendement_kg_ha',
                coalesce(col('rendement_kg_ha'), lit(0.0)))
)

# ── Pipeline MLlib ──────────────────────────────────────────────────────
NUM_COLS = [
    'ph_sol', 'pluviometrie_annuelle', 'temperature_moy_celsius',
    'humidite_relative_pct', 'teneur_matiere_organique_pct',
    'surface_hectares', 'stress_hydrique', 'ph_optimal_score',
    'indice_fertilite', 'annee'
]
CAT_COLS = ['region', 'culture', 'type_sol']

for c in NUM_COLS:
    gold_df = gold_df.withColumn(c, col(c).cast(DoubleType()))
gold_df = gold_df.withColumn('label', col('rendement_kg_ha').cast(DoubleType()))

indexers  = [StringIndexer(inputCol=c, outputCol=c+'_idx', handleInvalid='skip')
             for c in CAT_COLS]
encoders  = [OneHotEncoder(inputCols=[idx.getOutputCol()],
                           outputCols=[idx.getOutputCol().replace('_idx','_ohe')])
             for idx in indexers]
ohe_cols  = [enc.getOutputCols()[0] for enc in encoders]

assembler = VectorAssembler(
    inputCols=NUM_COLS + ohe_cols,
    outputCol='unscaled_features',
    handleInvalid='skip'
)
scaler    = StandardScaler(
    inputCol='unscaled_features', outputCol='features',
    withStd=True, withMean=True
)
rf = RandomForestRegressor(
    featuresCol='features', labelCol='label',
    numTrees=150, maxDepth=10, seed=42
)

if args.mode == 'train':
    ml_pipeline = Pipeline(stages=indexers + encoders + [assembler, scaler, rf])
    train, test = gold_df.randomSplit([0.8, 0.2], seed=42)
    model = ml_pipeline.fit(train)

    preds    = model.transform(test)
    ev_rmse  = RegressionEvaluator(labelCol='label',
                   predictionCol='prediction', metricName='rmse')
    ev_r2    = RegressionEvaluator(labelCol='label',
                   predictionCol='prediction', metricName='r2')
    rmse = ev_rmse.evaluate(preds)
    r2   = ev_r2.evaluate(preds)
    print(f'RandomForest | RMSE : {rmse:.2f} kg/ha | R² : {r2:.4f}')

    # Feature importance
    importances = model.stages[-1].featureImportances.toArray()
    names = NUM_COLS + CAT_COLS
```

```
    for nm, imp in sorted(zip(names[:len(importances)], importances),
                          key=lambda x: -x[1])[:8]:
        print(f'  {nm:40s} : {imp:.4f}')

    model.write().overwrite().save(MODEL_PATH)
    print(f'Modèle sauvegardé : {MODEL_PATH}')

else:  # mode infer
    model = PipelineModel.load(MODEL_PATH)
    preds = model.transform(gold_df)
    preds.select('plot_id_secure','region','culture','prediction') \
         .write.mode('overwrite').saveAsTable('agri_insight.predictions_gold')
    print(f'Prédictions écrites : {preds.count()} lignes')

spark.stop()
```

3.6 Catalogue Hive Bronze/Silver/Gold — hive_setup.sql

Architecture médaillon (Bronze/Silver/Gold) : Bronze = données brutes NiFi, Silver = données anonymisées + features dérivées partitionnées par région, Gold = prédictions. La vue vue_recommandations identifie automatiquement la CULTURE_OPTIMALE par région et type de sol via une fenêtre analytique.

hive_setup.sql

```
-- hive_setup.sql — Catalogue Bronze / Silver / Gold Agri-Insight SN

CREATE DATABASE IF NOT EXISTS agri_insight
    COMMENT 'Pipeline Agri-Insight SN — UADB 2025-2026';
USE agri_insight;

-- ── BRONZE : données brutes ingérées par NiFi ──────────────────────────
CREATE EXTERNAL TABLE IF NOT EXISTS parcelles_bronze (
    parcel_id                    STRING,
    producer_name                STRING,
    region                       STRING,
    culture                      STRING,
    type_sol                     STRING,
    ph_sol                       DOUBLE,
    pluviometrie_annuelle        DOUBLE,
    temperature_moy_celsius      DOUBLE,
    humidite_relative_pct        DOUBLE,
    teneur_matiere_organique_pct DOUBLE,
    surface_hectares             DOUBLE,
    annee                        INT,
    rendement_kg_ha              DOUBLE
)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
STORED AS TEXTFILE
LOCATION '/agri_insight/bronze/'
TBLPROPERTIES ('skip.header.line.count'='1');

-- ── SILVER : données anonymisées + features dérivées ───────────────────
CREATE TABLE IF NOT EXISTS parcelles_silver (
    plot_id_secure               STRING COMMENT 'SHA-256 — jamais parcel_id brut',
    region                       STRING,
    culture                      STRING,
    type_sol                     STRING,
    ph_sol                       DOUBLE,
    pluviometrie_annuelle        DOUBLE,
    temperature_moy_celsius      DOUBLE,
    humidite_relative_pct        DOUBLE,
    teneur_matiere_organique_pct DOUBLE,
    surface_hectares             DOUBLE,
    stress_hydrique              DOUBLE,
    ph_optimal_score             DOUBLE,
    indice_fertilite             DOUBLE,
    annee                        INT,
    rendement_kg_ha              DOUBLE
)
PARTITIONED BY (region_part STRING)
STORED AS ORC TBLPROPERTIES ('orc.compress'='SNAPPY');

-- ── GOLD : prédictions + recommandations ───────────────────────────────
CREATE TABLE IF NOT EXISTS predictions_gold (
    plot_id_secure   STRING,
    region           STRING,
    culture          STRING,
    rendement_predit DOUBLE,
    prediction_ts    TIMESTAMP
)
STORED AS ORC;

-- ── Vue recommandations par région et type de sol ──────────────────────
CREATE OR REPLACE VIEW vue_recommandations AS
SELECT
    s.region,
    s.type_sol,
```

```
    s.culture,
    AVG(COALESCE(p.rendement_predit, s.rendement_kg_ha)) AS rendement_moyen_predit,
    COUNT(*)                                              AS nb_parcelles,
    AVG(s.ph_sol)                                         AS ph_moyen,
    AVG(s.pluviometrie_annuelle)                          AS pluvio_moyenne,
    AVG(s.stress_hydrique)                                AS stress_hydrique_moyen,
    CASE
        WHEN AVG(p.rendement_predit) >=
             MAX(AVG(p.rendement_predit)) OVER (PARTITION BY s.region) * 0.9
        THEN 'CULTURE_OPTIMALE'
        WHEN AVG(p.rendement_predit) >=
             MAX(AVG(p.rendement_predit)) OVER (PARTITION BY s.region) * 0.7
        THEN 'CULTURE_VIABLE'
        ELSE 'CULTURE_DECONSEILLE'
    END AS statut_recommandation
FROM parcelles_silver s
LEFT JOIN predictions_gold p ON s.plot_id_secure = p.plot_id_secure
GROUP BY s.region, s.type_sol, s.culture
ORDER BY s.region, rendement_moyen_predit DESC;

-- ── Validation des volumes ──────────────────────────────────────────────
SELECT 'BRONZE' AS layer, COUNT(*) AS nb FROM parcelles_bronze
UNION ALL
SELECT 'SILVER', COUNT(*) FROM parcelles_silver
UNION ALL
SELECT 'GOLD',   COUNT(*) FROM predictions_gold;
```

3.7 DAG Airflow MLOps — agri_insight_mlops_dag.py

DAG hebdomadaire (lundi 3h). Le BranchPythonOperator calcule la MAE courante et décide : si MAE > 200 kg/ha → réentraîner le modèle, puis générer les recommandations. Les recommandations sont écrites directement dans HBase pour un accès temps réel.

dags/agri_insight_mlops_dag.py

```
# dags/agri_insight_mlops_dag.py — DAG MLOps Agri-Insight SN
# Réentraînement hebdomadaire + détection de drift + recommandations

from airflow import DAG
from airflow.operators.python import PythonOperator, BranchPythonOperator
from airflow.operators.dummy  import DummyOperator
from airflow.utils.dates      import days_ago
from datetime import timedelta
import subprocess, logging

logger = logging.getLogger('agri_insight_dag')
default_args = {
    'owner': 'seye_ahmed', 'retries': 2,
    'retry_delay': timedelta(minutes=10),
    'email_on_failure': True,
    'email': ['agri-insight@sonatel.sn'],
}

def compute_drift(**ctx):
    """Détecte le drift : compare RMSE courant vs RMSE de référence."""
    from pyhive import hive
    conn = hive.Connection(host='hive-metastore', port=10000)
    cursor = conn.cursor()
    cursor.execute("""
        SELECT
            AVG(ABS(rendement_predit - rendement_kg_ha)) AS mae_courant
        FROM agri_insight.predictions_gold g
        JOIN agri_insight.parcelles_silver s
          ON g.plot_id_secure = s.plot_id_secure
        WHERE g.prediction_ts >= DATE_SUB(CURRENT_DATE, 7)
    """)
    mae = cursor.fetchone()[0] or 0.0
```

```
    logger.info(f'MAE courante : {mae:.2f} kg/ha')
    ctx['ti'].xcom_push(key='mae', value=mae)
    # Seuil de drift : MAE > 200 kg/ha → réentraîner
    return 'retrain_model' if mae > 200 else 'generate_recommendations'

def retrain_model(**ctx):
    """Lance le réentraînement Spark via spark-submit."""
    result = subprocess.run([
        'spark-submit', '--master', 'spark://spark-master:7077',
        '/opt/airflow/models/yield_prediction.py',
        '--mode', 'train',
        '--input-table', 'agri_insight.parcelles_silver',
    ], capture_output=True, text=True, timeout=3600)
    if result.returncode != 0:
        raise Exception(f'Spark train failed: {result.stderr[-500:]}')
    logger.info('Modèle réentraîné avec succès ✓')

def generate_recommendations(**ctx):
    """Lance l'inférence et écrit les recommandations dans HBase."""
    from pyhive import hive
    import happybase
    from datetime import datetime
    # 1. Lancer inférence Spark
    subprocess.run([
        'spark-submit', '--master', 'spark://spark-master:7077',
        '/opt/airflow/models/yield_prediction.py',
        '--mode', 'infer',
    ], check=True, timeout=1800)
    # 2. Écrire les recommandations dans HBase
    conn_hbase = happybase.Connection('hbase', port=9090)
    conn_hbase.open()
    conn_hive = hive.Connection(host='hive-metastore', port=10000)
    cursor = conn_hive.cursor()
    cursor.execute("""
        SELECT region, type_sol, culture,
               rendement_moyen_predit, statut_recommandation
        FROM agri_insight.vue_recommandations
        WHERE statut_recommandation = 'CULTURE_OPTIMALE'
    """)
    table = conn_hbase.table(b'agri:recommandations')
    ts = datetime.utcnow().isoformat()
    for row in cursor.fetchall():
        region, sol, culture, rend, statut = row
        key = f'{region}_{sol}_{culture}'.encode()
        table.put(key, {
            b'meta:region':            region.encode(),
            b'meta:type_sol':          sol.encode(),
            b'predict:culture':        culture.encode(),
            b'predict:rendement':      str(round(rend,1)).encode(),
            b'conseil:statut':         statut.encode(),
            b'conseil:ts':             ts.encode(),
        })
    conn_hbase.close()
    logger.info('Recommandations HBase mises à jour ✓')

with DAG('agri_insight_mlops',
         default_args=default_args,
         description='MLOps Agri-Insight SN — drift + recommandations',
         schedule_interval='0 3 * * 1',  # Chaque lundi 3h
         start_date=days_ago(1), catchup=False,
         tags=['agri-insight','mlops','securite-alimentaire']) as dag:

    start    = DummyOperator(task_id='start')
    end      = DummyOperator(task_id='end')
    t_drift  = BranchPythonOperator(task_id='compute_drift',
                                    python_callable=compute_drift,
                                    provide_context=True)
    t_train  = PythonOperator(task_id='retrain_model',
                              python_callable=retrain_model,
                              provide_context=True)
    t_reco   = PythonOperator(task_id='generate_recommendations',
                              python_callable=generate_recommendations,
                              provide_context=True)
```

```
start >> t_drift >> [t_train, t_reco]
t_train >> t_reco >> end
```

3.8 Dashboard Recommandations — dashboard_recommandations.py

> ✓ **Livrable 4 — Dashboard**
> 4 panneaux : (1) Heatmap rendement prédit par culture×région, (2) Culture optimale par région, (3) Corrélation pluviométrie/rendement, (4) Indice stress hydrique. Les recommandations textuelles sont générées automatiquement à partir de vue_recommandations.

dashboard_recommandations.py

```
# dashboard_recommandations.py — Tableau de bord Agri-Insight SN
# Livrable 4 : Dashboard recommandations culture optimale par région

import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import seaborn as sns
from pyhive import hive

conn = hive.Connection(host='hive-metastore', port=10000, database='agri_insight')

# ── 1. Charger les recommandations ──────────────────────────────────────
df = pd.read_sql("""
    SELECT region, type_sol, culture,
           rendement_moyen_predit, nb_parcelles,
           ph_moyen, pluvio_moyenne,
           stress_hydrique_moyen, statut_recommandation
    FROM agri_insight.vue_recommandations
""", conn)

# ── 2. Dashboard 4 panneaux ─────────────────────────────────────────────
fig, axes = plt.subplots(2, 2, figsize=(16, 12))
fig.suptitle('Agri-Insight SN — Dashboard Recommandations Agricoles',
             fontsize=16, fontweight='bold', color='#1B4D1E')

# Panneau 1 : Rendement prédit par région (heatmap culture × région)
pivot = df.pivot_table(values='rendement_moyen_predit',
                       index='culture', columns='region', aggfunc='mean')
sns.heatmap(pivot, annot=True, fmt='.0f', cmap='YlGn',
            ax=axes[0,0], linewidths=0.5)
axes[0,0].set_title('Rendement prédit moyen (kg/ha) par culture et région')
axes[0,0].tick_params(axis='x', rotation=30)

# Panneau 2 : Culture optimale par région (barplot)
opt = df[df['statut_recommandation']=='CULTURE_OPTIMALE'] \
         .groupby(['region','culture'])['rendement_moyen_predit'].mean() \
         .reset_index()
colors = ['#16A34A','#D97706','#2563EB','#9333EA','#DC2626']
for i, (reg, grp) in enumerate(opt.groupby('region')):
    axes[0,1].barh(reg, grp['rendement_moyen_predit'].max(),
                   color=colors[i % len(colors)], label=grp['culture'].iloc[0])
axes[0,1].set_title('Rendement culture optimale par région')
axes[0,1].set_xlabel('Rendement prédit (kg/ha)')
axes[0,1].legend(title='Culture optimale', fontsize=8)

# Panneau 3 : Corrélation pluviométrie / rendement par culture
for i, culture in enumerate(df['culture'].unique()):
    sub = df[df['culture']==culture]
    axes[1,0].scatter(sub['pluvio_moyenne'], sub['rendement_moyen_predit'],
                      label=culture, alpha=0.7, s=60, color=colors[i%len(colors)])
axes[1,0].set_xlabel('Pluviométrie moyenne (mm/an)')
axes[1,0].set_ylabel('Rendement prédit (kg/ha)')
axes[1,0].set_title('Pluviométrie vs Rendement par culture')
axes[1,0].legend(fontsize=8)

# Panneau 4 : Impact du stress hydrique par région
stress_reg = df.groupby('region')['stress_hydrique_moyen'].mean().sort_values()
bars = axes[1,1].barh(stress_reg.index, stress_reg.values,
                      color=['#DC2626' if v < 0.7 else '#16A34A'
                             for v in stress_reg.values])
```

```
axes[1,1].axvline(x=1.0, color='black', linestyle='--', linewidth=1,
                  label='Besoin optimal (ratio=1)')
axes[1,1].set_title('Indice de stress hydrique moyen par région')
axes[1,1].set_xlabel('Ratio pluvio réelle / besoin culture')
axes[1,1].legend(fontsize=8)

plt.tight_layout()
plt.savefig('dashboard_agri_insight.png', dpi=150, bbox_inches='tight')
print('Dashboard sauvegardé : dashboard_agri_insight.png ✓')

# ── 3. Recommandations textuelles ──────────────────────────────────────
print('\n=== RECOMMANDATIONS AGRICOLES ===')
for _, row in df[df['statut_recommandation']=='CULTURE_OPTIMALE'].iterrows():
    print(f"Dans la région {row['region']} (sol {row['type_sol']},
           pH~{row['ph_moyen']:.1f}, {row['pluvio_moyenne']:.0f}mm/an) :")
    print(f"  → Privilégier le {row['culture']} :
           rendement prédit {row['rendement_moyen_predit']:.0f} kg/ha")
```

4. Guide de Test End-to-End

test_pipeline.sh

```
# test_pipeline.sh — Guide de test end-to-end Agri-Insight SN

# 1. Démarrage infrastructure
docker compose up -d zookeeper && sleep 10
docker compose up -d kafka nifi hbase hive-metastore namenode && sleep 30
docker compose up -d spark-master spark-worker airflow

# 2. Initialiser HBase
python hbase_setup.py
# Attendu : Table agri:recommandations créée ✓
#           Table agri:predictions créée ✓
#           Table agri:drift_monitor créée ✓

# 3. Générer les données
python generate_agri_data.py
# Attendu : Généré : ~4200 parcelles ✓

# 4. Vérifier les données dans HDFS
docker exec namenode hdfs dfs -ls /agri_insight/bronze/
docker exec namenode hdfs dfs -cat /agri_insight/bronze/agri_parcelles_brut.csv | head -3

# 5. Créer les tables Hive
docker exec hive-metastore hive -f hive_setup.sql
# Valider les volumes
docker exec hive-metastore beeline -u jdbc:hive2://localhost:10000 \
  -e 'SELECT * FROM agri_insight.vue_recommandations LIMIT 5;'

# 6. Entraîner le modèle
docker exec spark-master spark-submit \
  --master spark://spark-master:7077 \
  /opt/models/yield_prediction.py --mode train
# Attendu : RandomForest | RMSE : xxx kg/ha | R² : 0.xx

# 7. Lancer l'inférence
docker exec spark-master spark-submit \
  --master spark://spark-master:7077 \
  /opt/models/yield_prediction.py --mode infer

# 8. Vérifier les recommandations HBase
docker exec hbase hbase shell <<EOF
scan 'agri:recommandations', {LIMIT => 5}
EOF

# 9. Générer le dashboard
python dashboard_recommandations.py
# Attendu : dashboard_agri_insight.png sauvegardé ✓
```

5. Livrables Attendus

#	Livrable	Contenu attendu
1	**Pipeline de Données**	Code source complet + template NiFi exporté (GetFile→PutHDFS vers Bronze). Données visibles dans HDFS et table Hive Bronze interrogeable.
2	**Modèle ML & MLOps**	yield_prediction.py (train + infer), RMSE/R^2 documentés pour RF. DAG Airflow fonctionnel avec branchement drift. MLflow tracking optionnel.
3	**Catalogue Hive**	Tables Bronze/Silver/Gold créées. vue_recommandations retourne des lignes avec statut CULTURE_OPTIMALE pour chaque région.
4	**Dashboard Recommandations**	dashboard_agri_insight.png avec 4 panneaux + recommandations textuelles imprimées (ex : « Dans la région de Louga, privilégier le Mil »).

6. Barème de Notation — /20 points

> ℹ️ **Barème différencié 4 niveaux**
> Chaque critère est évalué selon 4 niveaux. La démonstration live de la vue_recommandations avec au moins une CULTURE_OPTIMALE par région est exigée en soutenance. Bonus documentation +2 pts.

Critère	Pts	Insuffisant (0)	Passable (50%)	Bien (75%)	Excellent (100%)
Infrastructure & Ingestion NiFi	3 pts	NiFi absent, HDFS vide	NiFi configuré mais 1 seul flux	GetFile→PutHDFS pour 1 CSV, Bronze partiellement chargée	3 flux NiFi opérationnels, Bronze complète (4 200+ lignes), HDFS visible
Validation & Privacy (Pandera/CDP)	4 pts	Pas de validation, PII en clair dans Hive	Schéma partiel OU anonymisation sans sel	Schéma complet + SHA-256 + sel, parcel_id non supprimé	Schéma + contrainte Riz≥600mm + drop(parcel_id,producer_name) + 0 PII en Silver/Gold
Feature Engineering & Spark MLlib	6 pts	Pas de Spark ni de features dérivées	Lecture Hive seule, pas de pipeline MLlib	Pipeline MLlib (OHE+scaler+RF) sans features dérivées	3 features dérivées (stress_hydrique, ph_optimal, indice_fertilite) + RMSE/R^2 documentés
Mirroring HBase/Hive & Analytics	4 pts	Ni HBase ni Hive Gold configurés	Tables créées, données partielles	Bronze/Silver/Gold créés, vue_recommandations vide	Vue_recommandations avec CULTURE_OPTIMALE par région + recommandations HBase en live
Recommandations Agricoles (Pertinence métier)	3 pts	Aucune recommandation produite	Recommandations génériques sans contexte régional	Recommandations par région, sans lien au pH ou type de sol	Recommandations actionnables : « Dans la zone de Louga, sol Dior, pH~6.5 → Mil (1050 kg/ha prédit) »
TOTAL	20 pts				

Bonus Documentation (jusqu'à +2 pts)

Élément	Points bonus
README.md avec architecture + docker compose up	+0.5 pt
Commentaires de code en français + docstrings	+0.5 pt
MLflow : tracking RMSE/R^2 sur au moins 2 runs	+0.5 pt
Démo live vue_recommandations sans erreur en soutenance	+0.5 pt

> ⚠️ **Note Importante — Pertinence Métier**
> Le projet doit impérativement aboutir à des conseils exploitables pour les agriculteurs. Un projet techniquement correct mais sans recommandations agricoles concrètes et contextualisées (région, pH, type de sol) est plafonné à 14/20. Exemple attendu : « Dans la zone de Louga, privilégier le Mil pour un pH entre 6.0 et 7.5 — rendement prédit : 920 kg/ha ».

Bon courage à toutes les équipes !

"Always be a solution, never a problem."

Mr Ahmed Ben Sidy Bouya SEYE | Senior Big Data & AI Engineer | Groupe Sonatel